Listas de chequeo para validación de documentación de diseño
GA4-220501095-AA4-EV03

Presentado por:

Luis Alejandro Muñoz Sierra

Presentado a:

Naín Zúñiga

Servicio Nacional de Aprendizaje – SENA

Programa de Formación:

Análisis y desarrollo de software

Ficha: 2977356

Medellín, 29 de agosto de 2025

Listas de Chequeo para Validación de Documentación de Diseño

Software de Gestión de Maquinaria Pesada

Introducción

La calidad en el desarrollo de software no es un resultado accidental, sino el producto de procesos sistemáticos de control y verificación. Las listas de chequeo son herramientas fundamentales en este proceso, ya que proporcionan un método estructurado y objetivo para validar que los productos generados cumplen con los estándares y requisitos definidos.

Este documento presenta los formatos de validación finales para la documentación de diseño del proyecto "MaquiMu". A diferencia del diseño conceptual de los instrumentos, este entregable constituye las listas de chequeo operativas, listas para ser aplicadas por el equipo de calidad o los revisores técnicos. Incluyen una metodología clara y una escala de evaluación para asegurar un proceso de revisión consistente y eficaz.

1. Metodología de Validación y Escala de Incidencias

Para garantizar una revisión homogénea y objetiva, se establece la siguiente escala de incidencias que será utilizada en todos los formatos de validación.

- **Escala de Incidencias:**
  - **C (Conforme):** El criterio evaluado cumple completamente con los estándares de calidad y los requisitos del proyecto. No se requieren acciones.
  - **NCM (No Conforme Menor):** El criterio presenta una desviación menor que no afecta la integridad del diseño (ej. error tipográfico, falta de claridad en una descripción). Requiere una corrección o aclaración simple.
  - **NCMy (No Conforme Mayor):** El criterio presenta un error u omisión significativa que compromete la calidad o consistencia del diseño (ej. un campo crítico faltante, una relación incorrecta). **Debe ser corregido** antes de que el artefacto pueda ser aprobado.

2. Formato de Validación: Modelo de Datos

Nombre del Instrumento: Lista de Chequeo para Validación del Modelo de Datos
Código: **`F-VAL-001`**

Artefacto a Verificar: `Arquitectura de software` **Versión:** 1.0

Fecha de Revisión: 30 - Agosto - 2025 **Revisor:** Jose Luis Muñoz Montañez

Resultado Final:(Aprobado)

Ítem	Criterio de Verificación	Resultado (C/NCM/NCMy)	Observaciones
1	¿El modelo conceptual identifica todas las entidades clave del negocio (**`Cliente`**, **`Maquinaria`**, **`Alquiler`**, **`Factura`**, **`Usuario`**)?	**C**	Sin observaciones
2	¿Las relaciones (1 a N) entre las entidades en el modelo lógico son correctas y están bien justificadas?	**C**	Sin observaciones
3	¿El modelo lógico demuestra estar normalizado hasta la Tercera Forma Normal (3FN)?	**C**	Sin observaciones
4	¿El diccionario de datos define claramente todos los campos, tipos de datos y restricciones (**PK**, **FK**, **`NOT NULL`**) para cada tabla?	**C**	Sin observaciones
5	¿Los tipos de datos seleccionados (ej. **`DECIMAL`**, **`VARCHAR`**, **`TIMESTAMP`**) son apropiados para la información que almacenarán?	**C**	Sin observaciones
6	¿Las políticas de seguridad (Integridad, Confidencialidad, Disponibilidad) están definidas de manera clara y aplicable?	**C**	Sin observaciones

3. Formato de Validación: Arquitectura de Software

Nombre del Instrumento: Lista de Chequeo para Validación de la Arquitectura de Software **Código:** `F-VAL-002`

Artefacto a Verificar: `Arquitectura de software` **Versión:** 1.0

Fecha de Revisión: 30 - Agosto - 2025 **Revisor:** Jose Luis Muñoz Montañez

Resultado Final:(Aprobado)

Ítem	Criterio de Verificación	Resultado (C/NCM/NCMy)	Observaciones
1	¿El patrón de arquitectura seleccionado (Hexagonal) está claramente explicado y se justifica su elección para el proyecto?	C	Sin observaciones
2	¿Se definen y explican patrones de diseño específicos (ej. Repositorio, DTO, Service Layer) que apoyan la arquitectura?	C	Sin observaciones
3	¿La vista de componentes identifica correctamente los módulos lógicos del sistema (**`Frontend`**, **`API`**, **`Núcleo`**, etc.) y sus dependencias?	C	Sin observaciones
4	¿La vista de despliegue representa de forma clara la infraestructura física/virtual (AWS, Docker, RDS) y la distribución de los componentes?	C	Sin observaciones
5	¿El documento lista las herramientas de software que se utilizarán para optimizar el ciclo de vida del desarrollo (CI/CD, Control de Versiones, etc.)?	C	Sin observaciones

4. Formato de Validación: Diagrama de Clases

Artefacto a Verificar: Diagrama de clases del proyecto **Versión:** 1.0
Artefacto a Verificar: Arquitectura de software **Versión:** 1.0
Fecha de Revisión: 30 - Agosto - 2025 **Revisor:** Jose Luis Muñoz Montañez

Resultado Final:(Aprobado)

Ítem	Criterio de Verificación	Resultado (C/NCM/NCMy)	Observaciones
1	¿El diagrama incluye todas las clases principales identificadas en la fase de análisis del sistema?	C	Sin observaciones
2	¿Cada clase define claramente sus atributos y métodos, aplicando principios de abstracción?	C	Sin observaciones
3	¿Se utilizan correctamente los modificadores de acceso (+ público, – privado) para promover el encapsulamiento?	C	Sin observaciones
4	¿Las relaciones entre clases (Asociación, Composición) están representadas de manera lógica y correcta?	C	Sin observaciones
5	¿El diagrama de clases es consistente con el modelo lógico de la base de datos?	C	Sin observaciones

5. Métrica de calidad del software

Artefacto a Verificar: `Métrica de calidad del software` **Versión:** 1.0

Fecha de Revisión: 30 - Agosto - 2025 **Revisor:** Jose Luis Muñoz Montañez

Resultado Final:(Aprobado)

Factor de Calidad	Criterio	Métrica Asociada	Cumple (Sí/No)
Operación del Producto *(Evalúa el software en ejecución)*	**Corrección** *(Cumple los requisitos)*	• Porcentaje de casos de prueba de aceptación superados. • Número de bugs reportados en producción por mes.	C
	Fiabilidad *(Funciona sin fallos)*	• Tiempo Medio Entre Fallos (MTBF). • Número de errores críticos (que bloquean la operación) por versión.	C
	Eficiencia *(Uso de recursos)*	• Tiempo de respuesta promedio de la API bajo carga normal. • Uso de CPU y memoria del contenedor Docker.	C
	Integridad *(Seguridad)*	• Número de vulnerabilidades detectadas por SonarQube. • Tiempo para detectar y resolver un intento de acceso no autorizado.	C
	Usabilidad *(Facilidad de uso)*	• Tiempo promedio para que un nuevo usuario complete una tarea clave (ej. crear un alquiler). • Puntuación de satisfacción del usuario (escala 1-5).	C
Revisión del Producto *(Evalúa la facilidad de modificación)*	**Mantenibilidad** *(Facilidad para corregir)*	• Complejidad ciclomática promedio del código. • Tiempo medio para resolver un bug desde su reporte.	C
	Flexibilidad *(Facilidad para añadir funciones)*	• Horas de desarrollo requeridas para implementar un cambio de alcance medio. • Número de clases que deben modificarse para agregar una nueva funcionalidad.	C
	Testeabilidad	• Porcentaje de cobertura de código de las pruebas unitarias.	C

	(Facilidad para probar)	• Tiempo necesario para ejecutar la suite completa de pruebas automatizadas.	
Transición del Producto *(Evalúa la adaptabilidad)*	**Portabilidad** *(Facilidad para migrar)*	• Tiempo para desplegar la aplicación en un nuevo entorno desde cero. • Número de dependencias específicas del sistema operativo (debería ser 0 gracias a Docker).	C
	Reusabilidad *(Facilidad para reutilizar código)*	• Número de componentes o módulos que se pueden usar en otros proyectos sin modificación.	C
	Interoperabilidad *(Facilidad para conectar)*	• Esfuerzo (en días) para integrar el sistema con una API de terceros (contabilidad).	C

6. Conclusiones

La estandarización del proceso de revisión a través de estas listas de chequeo es un paso crucial para el aseguramiento de la calidad del proyecto "MaquiMu". Estos formatos, diseñados para ser claros, concisos y fáciles de usar, permiten a los revisores evaluar de manera objetiva la documentación de diseño, identificar incidencias de forma sistemática y registrar las acciones correctivas necesarias.

Al implementar estas herramientas, se reduce la ambigüedad y la subjetividad en el proceso de validación, asegurando que todos los artefactos de diseño cumplan con un alto estándar de calidad antes de que comience la fase de construcción del software. Esto, a su vez, minimiza los riesgos, reduce los costos de corrección y contribuye directamente al éxito del proyecto.

HISTORIAL DE MODIFICACIONES

Versión	Naturaleza de cambio	fecha de aprobación	Fecha validación
1.0	Presentación inicial	2025-08-29	2025-08-30

FIRMAS

Elaboró	Aprobó	Validó
José Luis Muñoz Montañez	Inversionistas S.A.	Presidente MaquiMu
Secretaría MaquiMu	Inversionistas S.A.	Secretaria de turno.